

L19: Interval Analysis and Widening

Software Analysis

IIT Guwahati · September 22, 2026

Why Intervals?

Knowing the **range** of an integer variable at each program point catches a whole family of memory-safety bugs.

Why Intervals?

Knowing the **range** of an integer variable at each program point catches a whole family of memory-safety bugs.

- **Buffer overruns** — is the index within the allocated size?

Why Intervals?

Knowing the **range** of an integer variable at each program point catches a whole family of memory-safety bugs.

- **Buffer overruns** — is the index within the allocated size?
- **Array bounds** — C checks nothing for you.

Why Intervals?

Knowing the **range** of an integer variable at each program point catches a whole family of memory-safety bugs.

- **Buffer overruns** — is the index within the allocated size?
- **Array bounds** — C checks nothing for you.
- **Division by zero** — can the divisor's range contain 0?

Why Intervals?

Knowing the **range** of an integer variable at each program point catches a whole family of memory-safety bugs.

- **Buffer overruns** — is the index within the allocated size?
- **Array bounds** — C checks nothing for you.
- **Division by zero** — can the divisor's range contain 0?
- **Integer overflow** — can the value exceed the word size?

Why Intervals?

Knowing the **range** of an integer variable at each program point catches a whole family of memory-safety bugs.

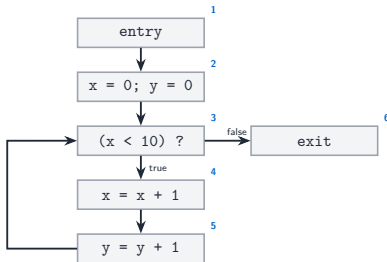
- **Buffer overruns** — is the index within the allocated size?
- **Array bounds** — C checks nothing for you.
- **Division by zero** — can the divisor's range contain 0?
- **Integer overflow** — can the value exceed the word size?

The ASTRÉE analyser is built on this, and is used on avionics and automotive code.

Interval Analysis

What Does It Compute?

For each program point and each integer variable, compute a **lower** and an **upper** bound on its value.

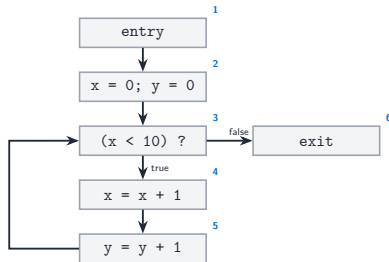


Interval Analysis

What Does It Compute?

For each program point and each integer variable, compute a **lower** and an **upper** bound on its value.

Two variables here: x and y .



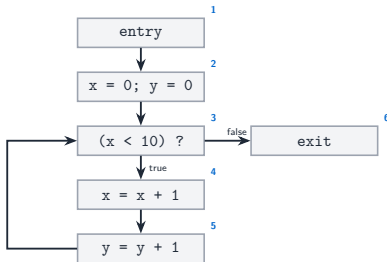
Interval Analysis

What Does It Compute?

For each program point and each integer variable, compute a **lower** and an **upper** bound on its value.

Two variables here: x and y .

The loop runs ten times — *but the analysis will not know that.*



Interval Analysis

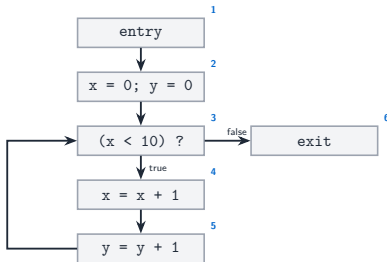
What Does It Compute?

For each program point and each integer variable, compute a **lower** and an **upper** bound on its value.

Two variables here: x and y .

The loop runs ten times — *but the analysis will not know that.*

Recall: conditions are abstracted by non-deterministic choice, so $x < 10$ is ignored.



Application 1: Detecting Integer Overflow

In C an `int` wraps round at the word size.

```
void overflow() {  
    char *out;  
    int in = get_int();  
    if (in <= 0) return;  
    out = malloc(in *  
                sizeof(char*));  
    for (i = 0; i < in; i++)  
        out[i] = get_string();  
}
```

Application 1: Detecting Integer Overflow

In C an `int` wraps round at the word size.

```
void overflow() {  
    char *out;  
    int in = get_int();  
    if (in <= 0) return;  
    out = malloc(in *  
                sizeof(char*));  
    for (i = 0; i < in; i++)  
        out[i] = get_string();  
}
```

The user types 1073741824.
Here `sizeof(char*)` is 4.

Application 1: Detecting Integer Overflow

In C an int wraps round at the word size.

```
void overflow() {  
    char *out;  
    int in = get_int();  
    if (in <= 0) return;  
    out = malloc(in *  
                sizeof(char*));  
    for (i = 0; i < in; i++)  
        out[i] = get_string();  
}
```

The user types 1073741824.

Here sizeof(char*) is 4.

$1073741824 \times 4 = 2^{32}$, wrapping round to **0**.

Application 1: Detecting Integer Overflow

In C an int wraps round at the word size.

```
void overflow() {  
    char *out;  
    int in = get_int();  
    if (in <= 0) return;  
    out = malloc(in *  
                sizeof(char*));  
    for (i = 0; i < in; i++)  
        out[i] = get_string();  
}
```

The user types 1073741824.

Here sizeof(char*) is 4.

$1073741824 \times 4 = 2^{32}$, wrapping round to **0**.

malloc(0) succeeds; the loop writes to pointers into a **0-byte** buffer.

Application 1: Detecting Integer Overflow

In C an `int` wraps round at the word size.

```
void overflow() {  
    char *out;  
    int in = get_int();  
    if (in <= 0) return;  
    out = malloc(in *  
                sizeof(char*));  
    for (i = 0; i < in; i++)  
        out[i] = get_string();  
}
```

The user types 1073741824.

Here `sizeof(char*)` is 4.

$1073741824 \times 4 = 2^{32}$, wrapping round to **0**.

`malloc(0)` succeeds; the loop writes to pointers into a **0-byte** buffer.

No guard: $\text{in} \in [-\infty, +\infty]$ — 0 possible.

Application 1: Detecting Integer Overflow

In C an `int` wraps round at the word size.

```
void overflow() {  
    char *out;  
    int in = get_int();  
    if (in <= 0) return;  
    out = malloc(in *  
                sizeof(char*));  
    for (i = 0; i < in; i++)  
        out[i] = get_string();  
}
```

The user types 1073741824.

Here `sizeof(char*)` is 4.

$1073741824 \times 4 = 2^{32}$, wrapping round to **0**.

`malloc(0)` succeeds; the loop writes to pointers into a **0-byte** buffer.

No guard: $\text{in} \in [-\infty, +\infty]$ — 0 possible.

With it: $\text{in} \in [1, +\infty]$.

Application 1: Detecting Integer Overflow

In C an `int` wraps round at the word size.

```
void overflow() {
    char *out;
    int in = get_int();
    if (in <= 0) return;
    out = malloc(in *
                sizeof(char*));
    for (i = 0; i < in; i++)
        out[i] = get_string();
}
```

The user types 1073741824.

Here `sizeof(char*)` is 4.

$1073741824 \times 4 = 2^{32}$, wrapping round to **0**.

`malloc(0)` succeeds; the loop writes to pointers into a **0-byte** buffer.

No guard: $\text{in} \in [-\infty, +\infty]$ — 0 possible.

With it: $\text{in} \in [1, +\infty]$.

CVE-2019-3855 (LibSSH) and **CVE-2019-8099** (Adobe Acrobat) are exactly this bug.

Application 2: Detecting Index-out-of-Bounds

C checks no array bound for you.

```
int main() {
    char *items[] =
        {"boat", "car",
         "truck", "train"};
    int index = get_int();
    if (index < 0 ||
        index > 3) return 1;
    printf("%s\n",
           items[index]);
}
```

Application 2: Detecting Index-out-of-Bounds

C checks no array bound for you.

```
int main() {  
    char *items[] =  
        {"boat", "car",  
         "truck", "train"};  
    int index = get_int();  
    if (index < 0 ||  
        index > 3) return 1;  
    printf("%s\n",  
           items[index]);  
}
```

Four entries, so the **safe range** for index is [0, 3].

Application 2: Detecting Index-out-of-Bounds

C checks no array bound for you.

```
int main() {
    char *items[] =
        {"boat", "car",
         "truck", "train"};
    int index = get_int();
    if (index < 0 ||
        index > 3) return 1;
    printf("%s\n",
           items[index]);
}
```

Four entries, so the **safe range** for index is $[0, 3]$.

Without the guard $\text{index} \in [-\infty, +\infty]$, which is $\not\subseteq [0, 3]$ — so warn.

Application 2: Detecting Index-out-of-Bounds

C checks no array bound for you.

```
int main() {
    char *items[] =
        {"boat", "car",
         "truck", "train"};
    int index = get_int();
    if (index < 0 ||
        index > 3) return 1;
    printf("%s\n",
           items[index]);
}
```

Four entries, so the **safe range** for index is $[0, 3]$.

Without the guard $\text{index} \in [-\infty, +\infty]$, which is $\not\subseteq [0, 3]$ — so warn.

With it $\text{index} \in [0, 3]$, which $\subseteq [0, 3]$ — so **proved safe**.

Application 2: Detecting Index-out-of-Bounds

C checks no array bound for you.

```
int main() {
    char *items[] =
        {"boat", "car",
         "truck", "train"};
    int index = get_int();
    if (index < 0 ||
        index > 3) return 1;
    printf("%s\n",
           items[index]);
}
```

Four entries, so the **safe range** for index is $[0, 3]$.

Without the guard $\text{index} \in [-\infty, +\infty]$, which is $\not\subseteq [0, 3]$ — so warn.

With it $\text{index} \in [0, 3]$, which $\subseteq [0, 3]$ — so **proved safe**.

Application 3: Detecting Divide-by-Zero

A division whose denominator is never checked.

```
int averageResponseTime(  
    int totalTime,  
    int numRequests) {  
    return totalTime  
        / numRequests;  
}
```


Application 3: Detecting Divide-by-Zero

A division whose denominator is never checked.

```
int averageResponseTime(  
    int totalTime,  
    int numRequests) {  
    return totalTime  
        / numRequests;  
}
```

The parameter is unconstrained:

numRequests $\in [-\infty, +\infty]$

0 lies inside it, so warn.

Application 3: Detecting Divide-by-Zero

A division whose denominator is never checked.

```
int averageResponseTime(  
    int totalTime,  
    int numRequests) {  
    return totalTime  
        / numRequests;  
}
```

The parameter is unconstrained:

numRequests $\in [-\infty, +\infty]$

0 lies inside it, so warn.

A server that divides by zero goes down
and stays down — a **denial of service**.

Application 3: Detecting Divide-by-Zero

A division whose denominator is never checked.

```
int averageResponseTime(  
    int totalTime,  
    int numRequests) {  
    return totalTime  
        / numRequests;  
}
```

The interval domain cannot say “non-zero”.

A guard `if (numRequests <= 0)`

gives the checker something it *can* use: $[1, +\infty]$.

The parameter is unconstrained:

$\text{numRequests} \in [-\infty, +\infty]$

0 lies inside it, so warn.

A server that divides by zero goes down
and stays down — a **denial of service**.

Application 3: Detecting Divide-by-Zero

A division whose denominator is never checked.

```
int averageResponseTime(  
    int totalTime,  
    int numRequests) {  
    return totalTime  
        / numRequests;  
}
```

The interval domain cannot say “non-zero”.

A guard `if (numRequests <= 0)`

gives the checker something it *can* use: $[1, +\infty]$.

The parameter is unconstrained:

$\text{numRequests} \in [-\infty, +\infty]$

0 lies inside it, so warn.

A server that divides by zero goes down and stays down — a **denial of service**.

CVE-2019-14498: a divide-by-zero in VLC, triggered by a crafted audio file.

All Three Checks Have the Same Shape

Operation at the node	Proved safe when
$a[i]$, array of size n	
y / x	
x stored in a machine word	

All Three Checks Have the Same Shape

Operation at the node	Proved safe when
$a[i]$, array of size n	$[l, h] \subseteq [0, n - 1]$
y / x	
x stored in a machine word	

All Three Checks Have the Same Shape

Operation at the node	Proved safe when
$a[i]$, array of size n	$[l, h] \subseteq [0, n - 1]$
y / x	$0 \notin [l, h]$
x stored in a machine word	

All Three Checks Have the Same Shape

Operation at the node	Proved safe when
$a[i]$, array of size n	$[l, h] \subseteq [0, n - 1]$
y / x	$0 \notin [l, h]$
x stored in a machine word	$[l, h] \subseteq [\text{INT_MIN}, \text{INT_MAX}]$

All Three Checks Have the Same Shape

Operation at the node	Proved safe when
$a[i]$, array of size n	$[l, h] \subseteq [0, n - 1]$
y / x	$0 \notin [l, h]$
x stored in a machine word	$[l, h] \subseteq [\text{INT_MIN}, \text{INT_MAX}]$

Containment holds $\Rightarrow$ **proved safe**, no warning.

All Three Checks Have the Same Shape

Operation at the node	Proved safe when
$a[i]$, array of size n	$[l, h] \subseteq [0, n - 1]$
y / x	$0 \notin [l, h]$
x stored in a machine word	$[l, h] \subseteq [\text{INT_MIN}, \text{INT_MAX}]$

Containment holds $\Rightarrow$ **proved safe**, no warning.

Containment fails $\Rightarrow$ the operation *may* be unsafe. That is a warning, not a bug — **false alarms live here.**

The Abstract Domain: Intervals

- An abstract value is an interval $[l, h]$, with $l \in \mathbb{Z} \cup \{-\infty\}$ and $h \in \mathbb{Z} \cup \{+\infty\}$.

The Abstract Domain: Intervals

- An abstract value is an interval $[l, h]$, with $l \in \mathbb{Z} \cup \{-\infty\}$ and $h \in \mathbb{Z} \cup \{+\infty\}$.
- Plus a **bottom** element $\perp$, read “*not an integer*”.

The Abstract Domain: Intervals

- An abstract value is an interval $[l, h]$, with $l \in \mathbb{Z} \cup \{-\infty\}$ and $h \in \mathbb{Z} \cup \{+\infty\}$.
- Plus a **bottom** element $\perp$, read “*not an integer*”.
- Ordered by **inclusion**:
 $[l_1, h_1] \sqsubseteq [l_2, h_2] \quad \text{iff} \quad l_1 \geq l_2 \quad \text{and} \quad h_1 \leq h_2.$

The Abstract Domain: Intervals

- An abstract value is an interval $[l, h]$, with $l \in \mathbb{Z} \cup \{-\infty\}$ and $h \in \mathbb{Z} \cup \{+\infty\}$.
- Plus a **bottom** element $\perp$, read “*not an integer*”.
- Ordered by **inclusion**:
$$[l_1, h_1] \sqsubseteq [l_2, h_2] \quad \text{iff} \quad l_1 \geq l_2 \quad \text{and} \quad h_1 \leq h_2.$$

A *narrower* interval is a stronger claim, so narrower is more precise.

The Abstract Domain: Intervals

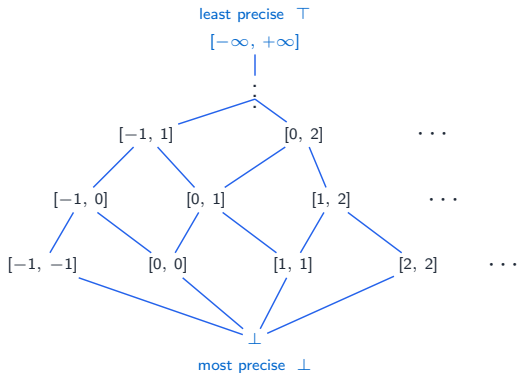
- An abstract value is an interval $[l, h]$, with $l \in \mathbb{Z} \cup \{-\infty\}$ and $h \in \mathbb{Z} \cup \{+\infty\}$.
- Plus a **bottom** element $\perp$, read “*not an integer*”.
- Ordered by **inclusion**:

$$[l_1, h_1] \sqsubseteq [l_2, h_2] \quad \text{iff} \quad l_1 \geq l_2 \quad \text{and} \quad h_1 \leq h_2.$$

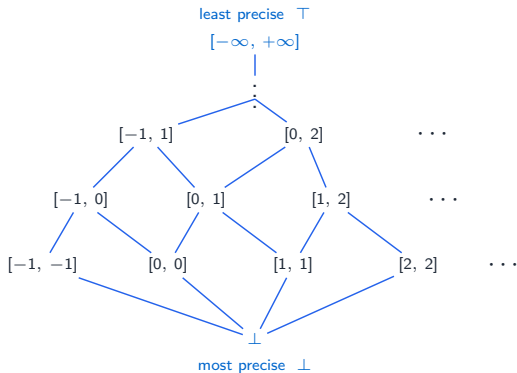
A *narrower* interval is a stronger claim, so narrower is more precise.

$\perp$ is the value at unreachable points, and the result of division by zero. Without it there would be no least element.

The Lattice — and It Is Infinite

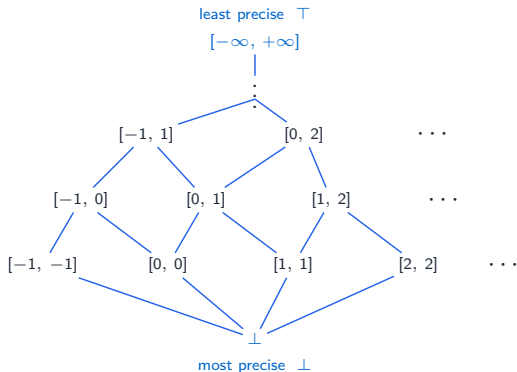


The Lattice — and It Is Infinite



This lattice has infinite height. $[0, 0] \sqsubset [0, 1] \sqsubset [0, 2] \sqsubset \dots$ never stops rising.

The Lattice — and It Is Infinite

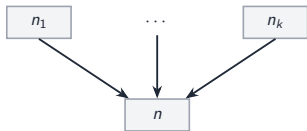


This lattice has infinite height. $[0, 0] \sqsubset [0, 1] \sqsubset [0, 2] \sqsubset \dots$ never stops rising.

Every termination argument so far assumed *finite* height.

Operation #1: Merge at the Entry

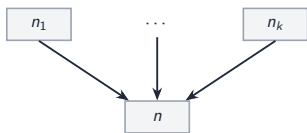
Interval analysis is a **forward** analysis, so the entry of n is built from the exits of its predecessors $n_1 \dots n_k$.



$$\text{IN}[n] = \bigsqcup_{n' \in \text{pred}(n)} \text{OUT}[n']$$

Operation #1: Merge at the Entry

Interval analysis is a **forward** analysis, so the entry of n is built from the exits of its predecessors $n_1 \dots n_k$.



$$\text{IN}[n] = \bigsqcup_{n' \in \text{pred}(n)} \text{OUT}[n']$$

and for *each* variable x , separately:

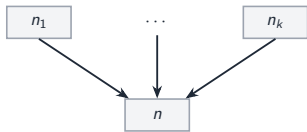
$$\text{IN}[n](x) = \left[\min(l_1 \dots l_k), \max(h_1 \dots h_k) \right]$$

where $\text{OUT}[n_1](x) = [l_1, h_1], \dots, \text{OUT}[n_k](x) = [l_k, h_k]$.

Operation #1: Merge at the Entry

Interval analysis is a **forward** analysis, so the entry of n is built from the exits of its predecessors $n_1 \dots n_k$.

$$\text{IN}[n] = \bigsqcup_{n' \in \text{pred}(n)} \text{OUT}[n']$$



and for *each* variable x , separately:

$$\text{IN}[n](x) = \left[\min(l_1 \dots l_k), \max(h_1 \dots h_k) \right]$$

where $\text{OUT}[n_1](x) = [l_1, h_1], \dots, \text{OUT}[n_k](x) = [l_k, h_k]$.

Smallest lower bound, *largest* upper bound — the widest of the incoming intervals.

Merge: A Worked Example

Node n has two predecessors. The program has two variables.

	x	y
OUT[n_1]	[0, 0]	[3, 5]
OUT[n_2]	[7, 7]	[1, 2]
IN[n]		

Merge: A Worked Example

Node n has two predecessors. The program has two variables.

	x	y
OUT[n_1]	[0, 0]	[3, 5]
OUT[n_2]	[7, 7]	[1, 2]
IN[n]	[0, 7]	

x : $\min(0, 7) = 0$, $\max(0, 7) = 7$.

Merge: A Worked Example

Node n has two predecessors. The program has two variables.

	x	y
OUT[n_1]	[0, 0]	[3, 5]
OUT[n_2]	[7, 7]	[1, 2]
IN[n]	[0, 7]	[1, 5]

x : $\min(0, 7) = 0$, $\max(0, 7) = 7$.

y : $\min(3, 1) = 1$, $\max(5, 2) = 5$ — the two bounds came from *different* predecessors.

Merge: A Worked Example

Node n has two predecessors. The program has two variables.

	x	y
OUT[n_1]	[0, 0]	[3, 5]
OUT[n_2]	[7, 7]	[1, 2]
IN[n]	[0, 7]	[1, 5]

x : $\min(0, 7) = 0$, $\max(0, 7) = 7$.

y : $\min(3, 1) = 1$, $\max(5, 2) = 5$ — the two bounds came from *different* predecessors.

$\perp$ is the identity: $\perp \sqcup [l, h] = [l, h]$, which is the first pass at a loop head.

Merge: A Worked Example

Node n has two predecessors. The program has two variables.

	x	y
$\text{OUT}[n_1]$	$[0, 0]$	$[3, 5]$
$\text{OUT}[n_2]$	$[7, 7]$	$[1, 2]$
$\text{IN}[n]$	$[0, 7]$	$[1, 5]$

x : $\min(0, 7) = 0$, $\max(0, 7) = 7$.

y : $\min(3, 1) = 1$, $\max(5, 2) = 5$ — the two bounds came from *different* predecessors.

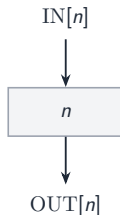
$\perp$ is the identity: $\perp \sqcup [l, h] = [l, h]$, which is the first pass at a loop head.

And precision is lost. x is 0 or 7 and nothing between, but $[0, 7]$ claims 3 is possible.

Operation #2: Transfer Through the Node

$\text{OUT}[n]$ from $\text{IN}[n]$. What happens depends on the statement at n .

$n: \boxed{x = y} \quad \text{OUT}[n](x) = \text{IN}[n](y)$



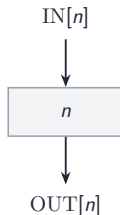
Operation #2: Transfer Through the Node

OUT[n] from IN[n]. What happens depends on the statement at n .

$$n: \boxed{x = y} \quad \text{OUT}[n](x) = \text{IN}[n](y)$$

$$n: \boxed{x = y - z} \quad \text{OUT}[n](x) = [y_1 - z_2, y_2 - z_1]$$

where $\text{IN}[n](y) = [y_1, y_2]$, $\text{IN}[n](z) = [z_1, z_2]$



Operation #2: Transfer Through the Node

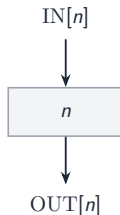
$\text{OUT}[n]$ from $\text{IN}[n]$. What happens depends on the statement at n .

$$n: \boxed{x = y} \quad \text{OUT}[n](x) = \text{IN}[n](y)$$

$$n: \boxed{x = y - z} \quad \text{OUT}[n](x) = [y_1 - z_2, y_2 - z_1]$$

$$\text{where } \text{IN}[n](y) = [y_1, y_2], \quad \text{IN}[n](z) = [z_1, z_2]$$

$$\text{OUT}[n](w) = \text{IN}[n](w) \quad \text{for every variable } w \text{ other than } x$$



Operation #2: Transfer Through the Node

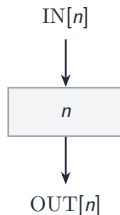
$\text{OUT}[n]$ from $\text{IN}[n]$. What happens depends on the statement at n .

$$n: \boxed{x = y} \quad \text{OUT}[n](x) = \text{IN}[n](y)$$

$$n: \boxed{x = y - z} \quad \text{OUT}[n](x) = [y_1 - z_2, y_2 - z_1]$$

where $\text{IN}[n](y) = [y_1, y_2]$, $\text{IN}[n](z) = [z_1, z_2]$

$\text{OUT}[n](w) = \text{IN}[n](w)$ for every variable w other than x



Subtraction crosses the bounds — the smallest y pairs with the largest z .

Transfer Rules, Statement by Statement

Entering n we know $y \in [1, 4]$ and $z \in [2, 3]$. The last column applies each rule to those.

Statement at n	OUT[n](x)	Example
$x = 7$		
$x = y$		
$x = y + z$		
$x = y - z$		

Transfer Rules, Statement by Statement

Entering n we know $y \in [1, 4]$ and $z \in [2, 3]$. The last column applies each rule to those.

Statement at n	OUT[n](x)	Example
$x = 7$	$[7, 7]$	$[7, 7]$
$x = y$		
$x = y + z$		
$x = y - z$		

Transfer Rules, Statement by Statement

Entering n we know $y \in [1, 4]$ and $z \in [2, 3]$. The last column applies each rule to those.

Statement at n	OUT[n](x)	Example
$x = 7$	$[7, 7]$	$[7, 7]$
$x = y$	IN[n](y)	$[1, 4]$
$x = y + z$		
$x = y - z$		

Transfer Rules, Statement by Statement

Entering n we know $y \in [1, 4]$ and $z \in [2, 3]$. The last column applies each rule to those.

Statement at n	OUT[n](x)	Example
$x = 7$	$[7, 7]$	$[7, 7]$
$x = y$	IN[n](y)	$[1, 4]$
$x = y + z$	$[y_1 + z_1, y_2 + z_2]$	$[1 + 2, 4 + 3] = [3, 7]$
$x = y - z$		

Transfer Rules, Statement by Statement

Entering n we know $y \in [1, 4]$ and $z \in [2, 3]$. The last column applies each rule to those.

Statement at n	OUT[n](x)	Example
$x = 7$	$[7, 7]$	$[7, 7]$
$x = y$	IN[n](y)	$[1, 4]$
$x = y + z$	$[y_1 + z_1, y_2 + z_2]$	$[1 + 2, 4 + 3] = [3, 7]$
$x = y - z$	$[y_1 - z_2, y_2 - z_1]$	$[1 - 3, 4 - 2] = [-2, 2]$

Transfer Rules, Statement by Statement

Entering n we know $y \in [1, 4]$ and $z \in [2, 3]$. The last column applies each rule to those.

Statement at n	OUT[n](x)	Example
$x = 7$	$[7, 7]$	$[7, 7]$
$x = y$	IN[n](y)	$[1, 4]$
$x = y + z$	$[y_1 + z_1, y_2 + z_2]$	$[1 + 2, 4 + 3] = [3, 7]$
$x = y - z$	$[y_1 - z_2, y_2 - z_1]$	$[1 - 3, 4 - 2] = [-2, 2]$

Addition keeps the bounds paired; subtraction swaps them. Compare $[3, 7]$ with $[-2, 2]$.

Multiplication

$$x = y * z \implies x \in [\min S, \max S]$$

Multiplication

$$x = y * z \implies x \in [\min S, \max S]$$

$$S = \{ y_1 z_1, y_1 z_2, y_2 z_1, y_2 z_2 \}$$

Multiplication

$$x = y * z \implies x \in [\min S, \max S]$$

$$S = \{ y_1 z_1, y_1 z_2, y_2 z_1, y_2 z_2 \}$$

Why all four? Signs decide which corner wins.

Multiplication

$$x = y * z \implies x \in [\min S, \max S]$$

$$S = \{ y_1 z_1, y_1 z_2, y_2 z_1, y_2 z_2 \}$$

Why all four? Signs decide which corner wins.

$$[-2, 3] \times [-4, 1] = [-12, 8] \quad \text{from } (3)(-4) \text{ and } (-2)(-4)$$

Multiplication

$$x = y * z \implies x \in [\min S, \max S]$$

$$S = \{ y_1 z_1, y_1 z_2, y_2 z_1, y_2 z_2 \}$$

Why all four? Signs decide which corner wins.

$$[-2, 3] \times [-4, 1] = [-12, 8] \quad \text{from } (3)(-4) \text{ and } (-2)(-4)$$

The same recipe, with more cases, handles division

Division

$$x = y / z \implies x \in [\min S, \max S]$$

Division

$$x = y / z \implies x \in [\min S, \max S]$$

$$S = \left\{ \left\lfloor \frac{y_1}{z_1} \right\rfloor, \left\lfloor \frac{y_1}{z_2} \right\rfloor, \left\lfloor \frac{y_2}{z_1} \right\rfloor, \left\lfloor \frac{y_2}{z_2} \right\rfloor \right\}$$

Division

$$x = y / z \implies x \in [\min S, \max S]$$

$$S = \left\{ \left\lfloor \frac{y_1}{z_1} \right\rfloor, \left\lfloor \frac{y_1}{z_2} \right\rfloor, \left\lfloor \frac{y_2}{z_1} \right\rfloor, \left\lfloor \frac{y_2}{z_2} \right\rfloor \right\}$$

But only if $0 \notin [z_1, z_2]$.

Division

$$x = y / z \implies x \in [\min S, \max S]$$

$$S = \left\{ \left\lfloor \frac{y_1}{z_1} \right\rfloor, \left\lfloor \frac{y_1}{z_2} \right\rfloor, \left\lfloor \frac{y_2}{z_1} \right\rfloor, \left\lfloor \frac{y_2}{z_2} \right\rfloor \right\}$$

But only if $0 \notin [z_1, z_2]$.

$$[6, 12] \div [2, 3] = [2, 6]$$

from $6/2, 6/3, 12/2, 12/3$

Division

$$x = y / z \implies x \in [\min S, \max S]$$

$$S = \left\{ \left\lfloor \frac{y_1}{z_1} \right\rfloor, \left\lfloor \frac{y_1}{z_2} \right\rfloor, \left\lfloor \frac{y_2}{z_1} \right\rfloor, \left\lfloor \frac{y_2}{z_2} \right\rfloor \right\}$$

But only if $0 \notin [z_1, z_2]$.

$$[6, 12] \div [2, 3] = [2, 6]$$

from $6/2, 6/3, 12/2, 12/3$

If $0 \in [z_1, z_2]$: **division by zero is possible**

Division

$$x = y / z \implies x \in [\min S, \max S]$$

$$S = \left\{ \left\lfloor \frac{y_1}{z_1} \right\rfloor, \left\lfloor \frac{y_1}{z_2} \right\rfloor, \left\lfloor \frac{y_2}{z_1} \right\rfloor, \left\lfloor \frac{y_2}{z_2} \right\rfloor \right\}$$

But only if $0 \notin [z_1, z_2]$.

$$[6, 12] \div [2, 3] = [2, 6]$$

from $6/2, 6/3, 12/2, 12/3$

If $0 \in [z_1, z_2]$: **division by zero is possible**

Conservatively: $x \in [-\infty, +\infty] = \top$

Quiz: Interval Arithmetic

Entering node n we know $y \in [1, 5]$ and $z \in [2, 3]$. What comes out?

1. n is $x = y - z$. Interval of x ?
2. Now $y \in [-3, 2]$, $z \in [2, 5]$, and n is $x = y * z$?
3. Two edges reach a merge, carrying $[0, 0]$ and $[5, 5]$. What does the merge produce?

Quiz: Interval Arithmetic

Entering node n we know $y \in [1, 5]$ and $z \in [2, 3]$. What comes out?

1. n is $x = y - z$. Interval of x ? $[-2, 3]$
2. Now $y \in [-3, 2]$, $z \in [2, 5]$, and n is $x = y * z$?
3. Two edges reach a merge, carrying $[0, 0]$ and $[5, 5]$. What does the merge produce?

Quiz: Interval Arithmetic

Entering node n we know $y \in [1, 5]$ and $z \in [2, 3]$. What comes out?

1. n is $x = y - z$. Interval of x ? $[-2, 3]$
2. Now $y \in [-3, 2]$, $z \in [2, 5]$, and n is $x = y * z$? $[-15, 10]$
3. Two edges reach a merge, carrying $[0, 0]$ and $[5, 5]$. What does the merge produce?

Quiz: Interval Arithmetic

Entering node n we know $y \in [1, 5]$ and $z \in [2, 3]$. What comes out?

1. n is $x = y - z$. Interval of x ? $[-2, 3]$
2. Now $y \in [-3, 2]$, $z \in [2, 5]$, and n is $x = y * z$? $[-15, 10]$
3. Two edges reach a merge, carrying $[0, 0]$ and $[5, 5]$. What does the merge produce? $[0, 5]$

Quiz: Interval Arithmetic

Entering node n we know $y \in [1, 5]$ and $z \in [2, 3]$. What comes out?

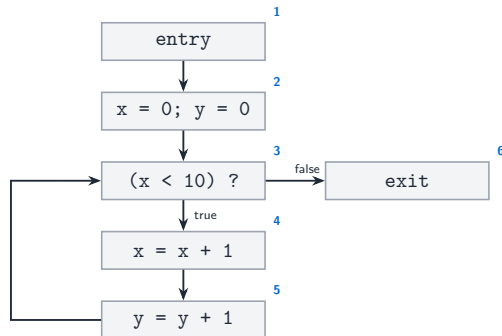
1. n is $x = y - z$. Interval of x ? $[-2, 3]$
2. Now $y \in [-3, 2]$, $z \in [2, 5]$, and n is $x = y * z$? $[-15, 10]$
3. Two edges reach a merge, carrying $[0, 0]$ and $[5, 5]$. What does the merge produce? $[0, 5]$

Passes 1 to 3

Init: every variable $\perp$ everywhere, except $\text{OUT}[1]$, where both are $[-\infty, +\infty]$.

	pass 1	pass 2	pass 3
OUT[3]			
OUT[4]			
OUT[5]			

Upper row x, lower row y.

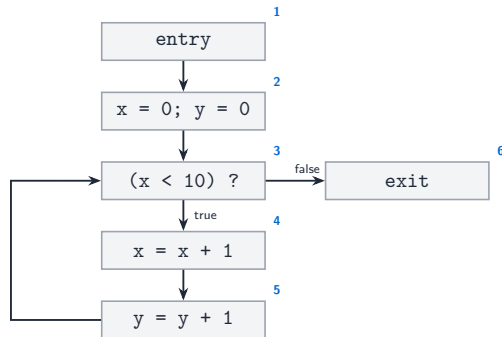


Passes 1 to 3

Init: every variable $\perp$ everywhere, except $\text{OUT}[1]$, where both are $[-\infty, +\infty]$.

	pass 1	pass 2	pass 3
OUT[3]	$[0, 0]$ $[0, 0]$		
OUT[4]	$[1, 1]$ $[0, 0]$		
OUT[5]	$[1, 1]$ $[1, 1]$		

Upper row x, lower row y.

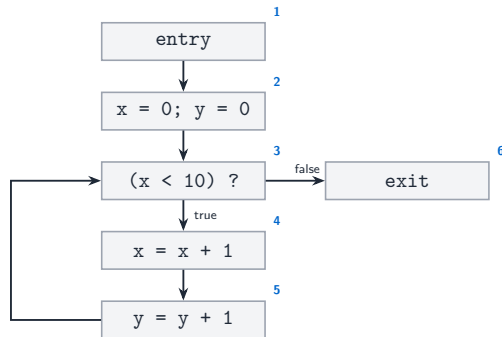


Passes 1 to 3

Init: every variable $\perp$ everywhere, except $\text{OUT}[1]$, where both are $[-\infty, +\infty]$.

	pass 1	pass 2	pass 3
OUT[3]	[0, 0] [0, 0]	[0, 1] [0, 1]	
OUT[4]	[1, 1] [0, 0]	[1, 2] [0, 1]	
OUT[5]	[1, 1] [1, 1]	[1, 2] [1, 2]	

Upper row x, lower row y.

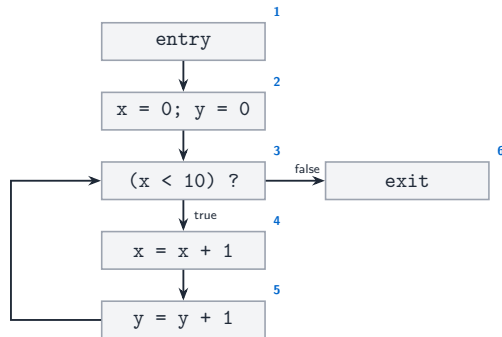


Passes 1 to 3

Init: every variable $\perp$ everywhere, except $\text{OUT}[1]$, where both are $[-\infty, +\infty]$.

	pass 1	pass 2	pass 3
OUT[3]	[0, 0] [0, 0]	[0, 1] [0, 1]	[0, 2] [0, 2]
OUT[4]	[1, 1] [0, 0]	[1, 2] [0, 1]	[1, 3] [0, 2]
OUT[5]	[1, 1] [1, 1]	[1, 2] [1, 2]	[1, 3] [1, 3]

Upper row x, lower row y.

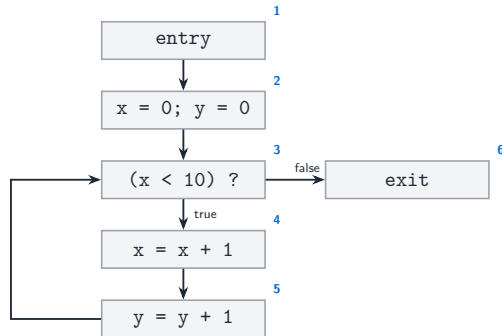


Passes 1 to 3

Init: every variable $\perp$ everywhere, except OUT[1],
where both are $[-\infty, +\infty]$.

	pass 1	pass 2	pass 3
OUT[3]	[0, 0] [0, 0]	[0, 1] [0, 1]	[0, 2] [0, 2]
OUT[4]	[1, 1] [0, 0]	[1, 2] [0, 1]	[1, 3] [0, 2]
OUT[5]	[1, 1] [1, 1]	[1, 2] [1, 2]	[1, 3] [1, 3]

Upper row x, lower row y.



Each pass analyses *one more iteration* of the loop — and every pass changes something.

Pass k : It Never Stops

	x	y
OUT[3]		
OUT[4]		
OUT[5]		

Pass k : It Never Stops

	x	y
OUT[3]	$[0, k - 1]$	$[0, k - 1]$
OUT[4]	$[1, k]$	$[0, k - 1]$
OUT[5]	$[1, k]$	$[1, k]$

Pass k : It Never Stops

	x	y
OUT[3]	$[0, k - 1]$	$[0, k - 1]$
OUT[4]	$[1, k]$	$[0, k - 1]$
OUT[5]	$[1, k]$	$[1, k]$

The chain of y at OUT[5]:

$$\perp \sqsubset [1, 1] \sqsubset [1, 2] \sqsubset [1, 3] \sqsubset \dots$$

Pass k : It Never Stops

	x	y
OUT[3]	$[0, k - 1]$	$[0, k - 1]$
OUT[4]	$[1, k]$	$[0, k - 1]$
OUT[5]	$[1, k]$	$[1, k]$

The chain of y at OUT[5]:

$$\perp \sqsubset [1, 1] \sqsubset [1, 2] \sqsubset [1, 3] \sqsubset \dots$$

An infinite strictly ascending chain. The iteration never saturates.

Pass k : It Never Stops

	x	y
OUT[3]	$[0, k - 1]$	$[0, k - 1]$
OUT[4]	$[1, k]$	$[0, k - 1]$
OUT[5]	$[1, k]$	$[1, k]$

The chain of y at OUT[5]:

$$\perp \sqsubset [1, 1] \sqsubset [1, 2] \sqsubset [1, 3] \sqsubset \dots$$

An infinite strictly ascending chain. The iteration never saturates. The limit *exists* — $[1, +\infty]$ — but no finite number of passes reaches it.

The Widening Operator

Compare the previous value with the new one. If a bound is **moving**, stop tracking it and jump straight to infinity.

$$[l_1, h_1] \nabla [l_2, h_2] = \left[l_2 < l_1 ? -\infty : l_1, \quad h_2 > h_1 ? +\infty : h_1 \right]$$

The Widening Operator

Compare the previous value with the new one. If a bound is **moving**, stop tracking it and jump straight to infinity.

$$[l_1, h_1] \nabla [l_2, h_2] = \left[l_2 < l_1 ? -\infty : l_1, \quad h_2 > h_1 ? +\infty : h_1 \right]$$

$$[1, 1] \nabla [1, 2] = [1, +\infty] \quad \text{upper bound grew, so give up on it}$$

The Widening Operator

Compare the previous value with the new one. If a bound is **moving**, stop tracking it and jump straight to infinity.

$$[l_1, h_1] \nabla [l_2, h_2] = \left[l_2 < l_1 ? -\infty : l_1, \quad h_2 > h_1 ? +\infty : h_1 \right]$$

$$[1, 1] \nabla [1, 2] = [1, +\infty] \quad \text{upper bound grew, so give up on it}$$

$$[1, 5] \nabla [1, 5] = [1, 5] \quad \text{nothing moved, so nothing is lost}$$

The Widening Operator

Compare the previous value with the new one. If a bound is **moving**, stop tracking it and jump straight to infinity.

$$[l_1, h_1] \nabla [l_2, h_2] = \left[l_2 < l_1 ? -\infty : l_1, \quad h_2 > h_1 ? +\infty : h_1 \right]$$

$$[1, 1] \nabla [1, 2] = [1, +\infty] \quad \text{upper bound grew, so give up on it}$$

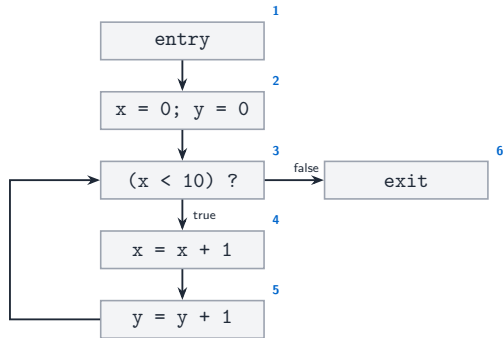
$$[1, 5] \nabla [1, 5] = [1, 5] \quad \text{nothing moved, so nothing is lost} \quad \text{Applied at } \mathbf{loop\ heads}$$

only — that is where the chains grow.

Widening Our Example

The chain of y at OUT[5], now widened:

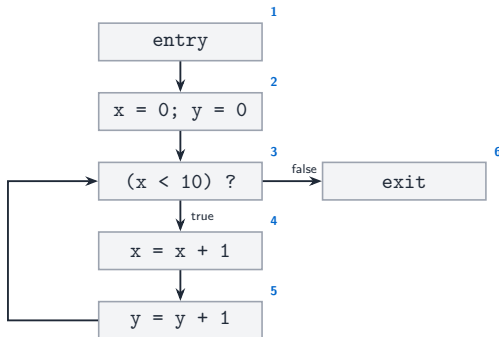
$$\perp \rightarrow [1, 1]$$



Widening Our Example

The chain of y at OUT[5], now widened:

$$\perp \rightarrow [1, 1] \rightarrow [1, 2]$$

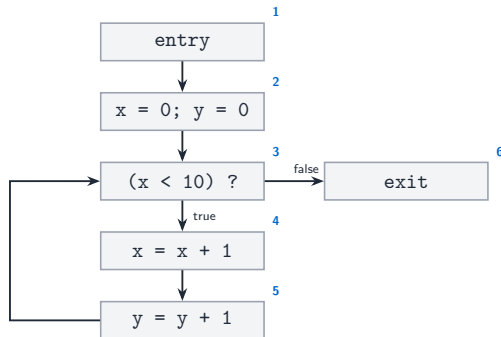


Widening Our Example

The chain of y at OUT[5], now widened:

$$\perp \rightarrow [1, 1] \rightarrow [1, 2] \rightarrow [1, +\infty]$$

upper bound moved



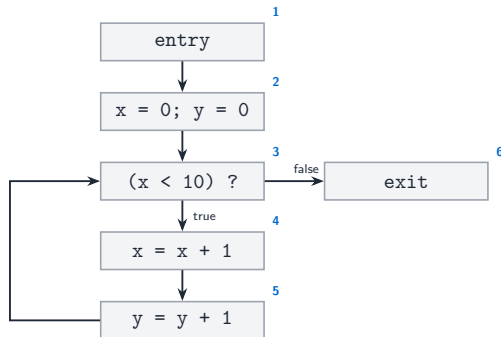
Widening Our Example

The chain of y at OUT[5], now widened:

$$\begin{array}{l} \perp \rightarrow [1, 1] \rightarrow [1, 2] \rightarrow [1, +\infty] \\ \rightarrow [1, +\infty] \end{array}$$

stable

upper bound moved



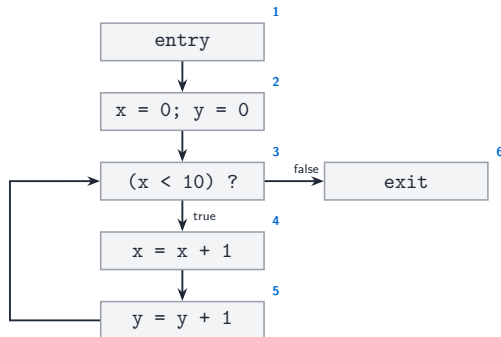
Widening Our Example

The chain of y at OUT[5], now widened:

$$\perp \rightarrow [1, 1] \rightarrow [1, 2] \rightarrow [1, +\infty]$$

upper bound moved

stable **Saturated** in a handful of passes.



Widening Our Example

The chain of y at OUT[5], now widened:

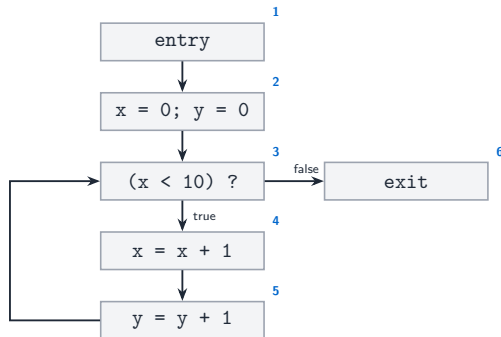
$$\perp \rightarrow [1, 1] \rightarrow [1, 2] \rightarrow [1, +\infty] \\ \rightarrow [1, +\infty]$$

upper bound moved

stable **Saturated** in a handful of passes.

Final: OUT[3] = ($x:[0, +\infty]$, $y:[0, +\infty]$)

OUT[5] = ($x:[1, +\infty]$, $y:[1, +\infty]$)



Widening Our Example

The chain of y at OUT[5], now widened:

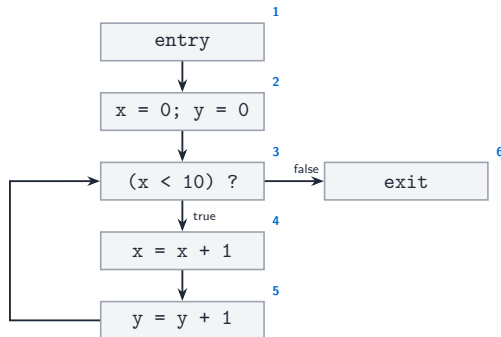
$$\perp \rightarrow [1, 1] \rightarrow [1, 2] \rightarrow [1, +\infty]$$

upper bound moved

stable **Saturated** in a handful of passes.

Final: OUT[3] = ($x:[0, +\infty]$, $y:[0, +\infty]$)

OUT[5] = ($x:[1, +\infty]$, $y:[1, +\infty]$)



Quiz: Widening

Apply $[l_1, h_1] \nabla [l_2, h_2]$: old value on the left, new value on the right.

1. $[2, 5] \nabla [1, 5] = ?$
2. $[0, 4] \nabla [-1, 7] = ?$
3. $[3, 7] \nabla [4, 6] = ?$

Quiz: Widening

Apply $[l_1, h_1] \nabla [l_2, h_2]$: old value on the left, new value on the right.

1. $[2, 5] \nabla [1, 5] = ?$ $[-\infty, 5]$
2. $[0, 4] \nabla [-1, 7] = ?$
3. $[3, 7] \nabla [4, 6] = ?$

Quiz: Widening

Apply $[l_1, h_1] \nabla [l_2, h_2]$: old value on the left, new value on the right.

1. $[2, 5] \nabla [1, 5] = ?$ $[-\infty, 5]$

2. $[0, 4] \nabla [-1, 7] = ?$ $[-\infty, +\infty]$

3. $[3, 7] \nabla [4, 6] = ?$

Quiz: Widening

Apply $[l_1, h_1] \nabla [l_2, h_2]$: old value on the left, new value on the right.

1. $[2, 5] \nabla [1, 5] = ?$ $[-\infty, 5]$

2. $[0, 4] \nabla [-1, 7] = ?$ $[-\infty, +\infty]$

3. $[3, 7] \nabla [4, 6] = ?$ $[3, 7]$

Quiz: Widening

Apply $[l_1, h_1] \nabla [l_2, h_2]$: old value on the left, new value on the right.

1. $[2, 5] \nabla [1, 5] = ?$ $[-\infty, 5]$

2. $[0, 4] \nabla [-1, 7] = ?$ $[-\infty, +\infty]$

3. $[3, 7] \nabla [4, 6] = ?$ $[3, 7]$

Widening only ever *widens*: question 3 keeps the old bounds rather than adopting the narrower new ones.

What Widening Costs

- The true answer is $x \in [0, 10]$ — the loop runs ten times.

What Widening Costs

- The true answer is $x \in [0, 10]$ — the loop runs ten times.
- We computed $x \in [0, +\infty]$. **Sound, but weak.**

What Widening Costs

- The true answer is $x \in [0, 10]$ — the loop runs ten times.
- We computed $x \in [0, +\infty]$. **Sound, but weak.**
- Two separate causes: the condition $x < 10$ was abstracted away, *and* widening jumped to $+\infty$.

What Widening Costs

- The true answer is $x \in [0, 10]$ — the loop runs ten times.
- We computed $x \in [0, +\infty]$. **Sound, but weak.**
- Two separate causes: the condition $x < 10$ was abstracted away, *and* widening jumped to $+\infty$.
- **Narrowing**: re-run from the widened result to tighten bounds.

What Widening Costs

- The true answer is $x \in [0, 10]$ — the loop runs ten times.
- We computed $x \in [0, +\infty]$. **Sound, but weak.**
- Two separate causes: the condition $x < 10$ was abstracted away, *and* widening jumped to $+\infty$.
- **Narrowing**: re-run from the widened result to tighten bounds.

Like every other operator in the framework, a widening operator is a *design choice* — and it trades precision for termination.